Multistage networks provide a compromise between the two extremes. The major advantage of MINs lies in their scalability with modular construction. However, the latency increases with log n, the number of stages in the network. Also, costs due to increased wiring and switching complexity are another constraint.

For building MPP systems, some of the static topologies are more scalable in specific applications. Advances in VLSI and interconnect technologies have had a major impact on multiprocessor system architecture, as we shall see in Chapter 13, and there has been a clear shift towards the use of packet-based switched-media interconnects.

Summary

LO 1 In this chapter, we have focused on basic program properties which make parallelism possible and determine the amount and type of parallelism which can be exploited. With increasing degree of multiprocessing, the rate at which data must be communicated between subsystems also increases, and therefore the system interconnect architecture becomes important in determining system performance.

We started this chapter with a study of the basic conditions which must be satisfied for parallel computations to be possible. In essence, it is dependences between operations which limit the amount of parallelism which can be exploited. After all, any set of N fully independent operations can always be performed in parallel.

The three basic data dependences between operations are *flow dependence*, *anti-dependence* and *output dependence*. *Resource dependence* refers to a limitation in available hardware and/or software resources which limits the achievable degree of parallelism. Bernstein's conditions—which apply to input and output sets of processes—must be satisfied for parallel execution of processes to be possible.

LO 2 Parallelism may be exploited at the level of software or hardware. For software parallelism, program design, and the program development and runtime environments play the key role. For hardware parallelism, availability of the right mix of hardware resources plays the key role. Program partitioning, grain size, communication latency and scheduling are important concepts; scheduling may be static or dynamic.

LO 3 Program flow may be control-driven, data-driven or demand-driven. Of these, control-driven program flow, as exemplified in the von Neumann model, is the only one that has proved commercially successful over the last six decades. Other program flow models have been tried out on research-oriented systems, but in general these models have not found acceptance on a broader basis.

LO 4 When computer systems consist of multiple processors—and several other sub-systems such as memory modules and network adapters—the system interconnect architecture plays a very important role in determining final system performance. We studied basic network properties, including topology and routing functionality. Network performance can be characterized in terms of bandwidth, latency, functionality, and scalability.

Contd.

We studied static network topologies such as the linear array, ring, tree, fat tree, torus and hypercube; we also looked at dynamic network topologies which involve switching and/or routing of data. With higher degree of multiprocessing, bus-based systems are unable to meet aggregate bandwidth requirements of the system; multistage interconnection networks and crossbar switches can provide better alternatives.

Exercises

Problem 2.1 Define the following terms related to parallelism and dependence relations:

(a) Computational granularity.
(b) Communication latency.
(c) Flow dependence.
(d) Antidependence.
(e) Output dependence.
(f) I/O dependence.
(g) Control dependence.
(h) Resource dependence.
(i) Bernstein conditions.
(j) Degree of parallelism.

Problem 2.2 Define the following terms for various system interconnect architectures:

(a) Node degree.
(b) Network diameter.
(c) Bisection bandwidth.
(d) Static connection networks.
(e) Dynamic connection networks.
(f) Nonblocking networks.
(g) Multicast and broadcast.
(h) Mesh versus torus.
(i) Symmetry in networks.
(j) Multistage networks.
(k) Crossbar networks.
(l) Digital buses.

Problem 2.3 Answer the following questions on program flow mechanisms and computer models:

(a) Compare control-flow, dataflow, and reduction computers in terms of the program flow mechanism used.
(b) Comment on the advantages and disadvantages in control complexity, potential for parallelism, and cost-effectiveness of the above computer models.
(c) What are the differences between string reduction and graph reduction machines?

Problem 2.4 Perform a data dependence analysis on each of the following Fortran program fragments. Show the dependence graphs among the statements with justification.

(a)
S1: A = B + D
S2: C = A × 3
S3: A = A + C
S4: E = A / 2

(b)
S1: X = SIN(Y)
S2: Z = X + W
S3: Y = –2.5 × W
S4: X = COS(Z)

(c) Determine the data dependences in the same and adjacent iterations of the following Do-loop.

```
      Do 10 I = 1, N
S1:      A(I + 1) = B(I – 1) + C(I)
S2:      B(I) = A(I) × K
S3:      C(I) = B(I) – 1
   10 Continue
```

Problem 2.5 Analyze the data dependences among the following statements in a given program:

```
S1: Load R1, 1024              /R1 ← 1024/
S2: Load R2, M(10)             /R2 ← Memory(10)/
S3: Add R1, R2                 /R1 ← (R1) + (R2)/
S4: Store M(1024), R1          /Memory(1024) ← (R1)/
S5: Store M((R2)), 1024        /Memory(64) ← 1024/
```

where (Ri) means the content of register Ri and Memory(10) contains 64 initially.

(a) Draw a dependence graph to show all the dependences.

(b) Are there any resource dependences if only one copy of each functional unit is available in the CPU?

(c) Repeat the above for the following program statements:

```
S1: Load R1, M(100)   /R1 ← Memory(100)/
S2: Move R2, R1       /R2 ← (R1)/
S3: Inc R1            /R1 ← (R1) + 1/
S4: Add R2, R1        /R2 ← (R2) + (R1)/
S5: Store M(100), R1  /Memory(100) ← (R1)/
```

Problem 2.6 A sequential program consists of the following five statements, S1 through S5. Considering each statement as a separate process, clearly identify *input set* I_i and *output set* O_i of each process. Restructure the program using Bernstein's conditions in order to achieve maximum parallelism between processes. If any pair of processes cannot be executed concurrently, specify which of the three conditions is not satisfied.


```
S1:  A = B + C
S2:  C = B × D
S3:  S = 0
S4:  Do I = A, 100
        S = S + X(I)
     End Do
S5:  IF (S .GT. 1000) C = C × 2
```

Problem 2.7 Consider the execution of the following code segment consisting of seven statements. Use Bernstein's conditions to detect the maximum parallelism embedded in this code. Justify the portions that can be executed in parallel and the remaining portions that must be executed sequentially. Rewrite the code using parallel constructs such as *Cobegin* and *Coend*. No variable substitution is allowed. All statements can be executed in parallel if they are declared within the same block of a (*Cobegin*, *Coend*) pair.

```
S1:  A = B + C
S2:  C = D + E
S3:  F = G + E
S4:  C = A + F
S5:  M = G + C
S6:  A = L + C
S7:  A = E + A
```

Problem 2.8 According to program order, the following six arithmetic expressions need to be executed in minimum time. Assume that all are integer operands already loaded into working registers. No memory reference is needed for the operand fetch. Also, all intermediate or final results are written back to working registers without conflicts.

```
P1:  X ← (A + B) × (A – B)
P2:  Y ← (C + D) / (C – D)
P3:  Z ← X + Y
P4:  A ← E × F
P5:  Y ← E – Z
P6:  B ← (X – F) × A
```

(a) Use the minimum number of working registers to rewrite the above HLL program into a minimum-length assembly language code using arithmetic opcodes *add*, *subtract*, *multiply*, and *divide* exclusively. Assume a fixed instruction format with three register fields: two for sources and one for destinations.

(b) Perform a flow analysis of the assembly code obtained in part (a) to reveal all data dependences with a dependence graph.

(c) The CPU is assumed to have two *add units*, one *multiply unit*, and one *divide unit*. Work out an optimal schedule to execute the assembly code in minimum time, assuming 1 cycle for the add unit, 3 cycles for the multiply unit, and 18 cycles for the divide unit to complete the execution

of one instruction. Ignore all overhead caused by instruction fetch, decode, and writeback. No pipelining is assumed here.

Problem 2.9 Consider the following assembly language code. Exploit the maximum degree of parallelism among the 16 instructions, assuming no resource conflicts and multiple functional units are available simultaneously. For simplicity, no pipelining is assumed. All instructions take one machine cycle to execute. Ignore all other overhead.

```
1:  Load R1, A          /R1 ← Mem(A)/
2:  Load R2, B          /R2 ← Mem(B)/
3:  Mul R3, R1, R2      /R3 ← (R1) × (R2)/
4:  Load R4, D          /R4 ← Mem(D)/
5:  Mul R5, R1, R4      /R5 ← (R1) × (R4)/
6:  Add R6, R3, R5      /R6 ← (R3) + (R5)/
7:  Store X, R6         /Mem(X) ← (R6)/
8:  Load R7, C          /R7 ← Mem(C)/
9:  Mul R8, R7, R4      /R8 ← (R7) × (R4)/
10: Load R9, E          /R9 ← Mem(E)/
11: Add R10, R8, R9     /R10 ← (R8) + (R9)/
12: Store Y, R10        /Mem(Y) ← (R10)/
13: Add R11, R6, R10    /R11 ← (R6) + (R10)/
14: Store U, R11        /Mem(U) ← (R11)/
15: Sub R12, R6, R10    /R12 ← (R6) – (R10)/
16: Store V, R12        /Mem(V) ← (R12)/
```

(a) Draw a program graph with 16 nodes to show the flow relationships among the 16 instructions.

(b) Consider the use of a three-issue superscalar processor to execute this program fragment in minimum time. The processor can issue one memory-access instruction (Load or Store but not both), one Add/Sub instruction, and one Mul (multiply) instruction per cycle. The Add unit, Load/Store unit, and Multiply unit can be used simultaneously if there is no data dependence.

Problem 2.10 Repeat part (b) of Problem 2.9 on a dual-processor system with shared memory. Assume that the same superscalar processors are used and that all instructions take one cycle to execute.

(a) Partition the given program into two balanced halves. You may want to insert some load or store instructions to pass intermediate results generated by the two processors to each other. Show the divided program flow graph with the final output U and V generated by the two processors separately.

(b) Work out an optimal schedule for parallel execution of the above divided program by the two processors in minimum time.

Problem 2.11 You are asked to design a direct network for a multicomputer with 64 nodes using a three-dimensional torus, a six-dimensional binary hypercube, and cube-connected-cycles (CCC) with a minimum diameter. The following questions are related to the relative merits of these network topologies:

(a) Let d be the node degree, D the network diameter, and l the total number of links in a network. Suppose the quality of a network is measured by $(d \times D \times l)^{-1}$. Rank the three architectures according to this quality measure.

(b) A *mean internode distance* is defined as the average number of hops (links) along the shortest path for a message to travel from one node to another. The average is calculated for all (source, destination) pairs. Order the three architectures based on their mean internode distances, assuming that the probability that a node will send a message to all other nodes with distance i is $(D - i + 1)/\Sigma_{k=1}^{D} k$, where D is the network diameter.

Problem 2.12 Consider an Illiac mesh (8×8), a binary hypercube, and a barrel shifter, all with 64 nodes labeled $N_0, N_1, \ldots, N_{63}$. All network links are bidirectional.

(a) List all the nodes reachable from node N_0 in exactly three steps for each of the three networks.

(b) Indicate in each case the tightest upper bound on the minimum number of routing steps needed to send data from any node N_i to another node N_j.

(c) Repeat part (b) for a larger network with 1024 nodes.

Problem 2.13 Compare *buses*, *crossbar switches*, and *multistage networks* for building a multiprocessor system with n processors and m shared-memory modules. Assume a word length of w bits and that 2×2 switches are used in building the multistage networks. The comparison study is carried out separately in each of the following four categories:

(a) Hardware complexities such as switching, arbitration, wires, connector, or cable requirements.

(b) Minimum latency in unit data transfer between the processor and memory module.

(c) Bandwidth range available to each processor.

(d) Communication capabilities such as permutations, data broadcast, blocking handling, etc.

Problem 2.14 Answer the following questions related to multistage networks:

(a) How many legitimate states are there in a 4×4 switch module, including both broadcast and permutations? Justify your answer with reasoning.

(b) Construct a 64-input Omega network using 4×4 switch modules in multiple stages. How many permutations can be implemented directly in a single pass through the network without blocking?

(c) What is the percentage of one-pass permutations compared with the total number of permutations achievable in one or more passes through the network?

Problem 2.15 Topologically equivalent networks are those whose graph representations are isomorphic with the same interconnection capabilities. Prove the topological equivalence among the Omega, Flip, and Baseline networks.

(a) Prove that the Omega network (Fig. 2.24) is topologically equivalent to the Baseline network (Fig. 2.25b).

(b) The *Flip network* (Fig. 2.27) is constructed using inverse perfect shuffle (Fig. 2.14b) for interstage connections. Prove that the Flip network is topologically equivalent to the Baseline network.

(c) Based on the results obtained in (a) and (b), prove the topological equivalence between the Flip network and the Omega network.

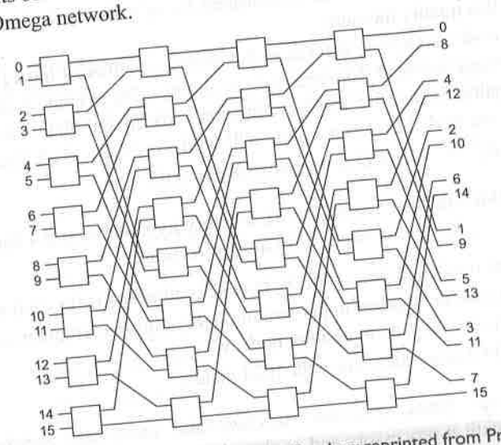

Fig. 2.27 A 16 × 16 Flip network (Courtesy of Ken Batcher; reprinted from Proc. Int. Conf. Parallel Processing, 1976)

Problem 2.16 Answer the following questions for the k-ary n-cube network:

(a) How many nodes does the network contain?
(b) What is the network diameter?
(c) What is the bisection bandwidth?
(d) What is the node degree?
(e) Explain the graph-theoretic relationship among k-ary n-cube networks and rings, meshes, tori, binary n-cubes, and Omega networks.
(f) Explain the difference between a conventional torus and a folded torus.
(g) Under the assumption of constant wire bisection, why do low-dimensional networks (tori) have lower latency and higher hot-spot throughput than high-dimensional networks (hypercubes)?

Problem 2.17 Read the paper on fat trees by Leiserson, which appeared in *IEEE Trans. Computers*, pp. 892–901, Oct. 1985. Answer the following questions related to the organization and application of fat trees:

(a) Explain the advantages of using binary fat trees over conventional binary trees as a multiprocessor interconnection network.
(b) A *universal fat tree* is defined as a fat tree of n nodes with root capacity w, where $n^{2/3} \le w \le n$, and for each channel c_k at level k of the tree, the capacity is

$$c_k = \min\left(\lceil n/2^k \rceil, \lceil w/2^{2k/3} \rceil\right)$$

Prove that the capacities of a universal fat tree grow exponentially as we go up the tree from the leaves. The channel capacity is defined here as the number of wires in a channel.

Problem 2.18 Read the paper on k-ary n-cube networks by Dally, which appeared in IEEE Trans. Computers, June 1990, pp. 775–785. Answer the following questions related to the network properties and applications as a VLSI communication network:

(a) Prove that the bisection width B of a k-ary n-cube with w-bit wide communication channels is

$$B(k, n) = 2w\sqrt{N}k^{n/2-1} = 2wN/k$$

where $N = k^n$ is the network size.

(b) Prove that the hot-spot throughput of a k-ary n-cube network with deterministic routing is equal to the bandwidth of a single channel $w = k - 1$, under the assumption of a constant wire cost.

Problem 2.19 Network embedding is a technique to implement a network A on a network B. Explain how to perform the following network embeddings:

(a) Embed a two-dimensional torus $r \times r$ on an n-dimensional hypercube with $N = 2^n$ nodes where $r^2 = 2^n$.
(b) Embed the largest ring on a CCC with $N = k \times 2^k$ nodes and $k \ge 3$.
(c) Embed a complete balanced binary tree with maximum height on a mesh of $r \times r$ nodes.

Problem 2.20 Read the paper on hypernets by Hwang and Ghosh, which appeared in *IEEE Trans. Computers*, Dec. 1989. Answer the following questions related to the network properties and applications of hypernets:

(a) Explain how hypernets integrate positive features of hypercube and tree-based topologies into one combined architecture.
(b) Prove that the average node degree of a hypernet can be maintained essentially constant when the network size is increased.
(c) Discuss the application potentials of hypernets in terms of message routing complexity, cost-effective support for global as well as localized communication, I/O capabilities, and fault tolerance.